1. Общие требования

Курс предусматривает два домашних задания. Домашнее задание №1 посвящено кодированию источников, домашнее задание №2 — помехоустойчивому кодированию; оно будет опубликовано отдельно. Каждое задание сдаётся в два приёма: сначала работа загружается в репозиторий, затем защищается лично у доски.

Каждое задание оценивается по двум отдельным критериям: **задание** — по сданным файлам, **защита** — по результатам защиты у доски (п. 1.7.). В критериях оценивания курса им соответствуют активности «Домашнее задание» и «Устный опрос»; веса в итоговой оценке указаны там. Сданная работа — условие допуска к защите, а защита — обязательное условие: задание без пройденной защиты не засчитывается (п. 1.7.).

1.1. РЕАЛИЗАЦИЯ

1. Язык программирования — Python версии 3.11 и выше, стандартная библиотека. Внешние зависимости допускаются только для построения графиков. Использование готовых реализаций кодов, конечных полей, энтропии, кодов Хаффмана и алгоритмов декодирования запрещено: эти части и проверяются. При выборе другого языка программирования требуется приложить один скрипт сборки, который собирает проект из чистого каталога одной командой, и указать проверенную версию компилятора.
2. Все режимы работы запускаются неинтерактивно — параметрами командной строки и файлами. Интерактивный диалог с пользователем допускается как дополнительный режим, но не как единственный. Это требование обязательно: на защите программа запускается на файлах преподавателя без правки исходного кода.
3. Любая случайность управляется параметром `--seed`. Два запуска с одинаковым seed дают побитово одинаковый результат.
4. Программа не должна завершаться с необработанным исключением ни на одном входе. Некорректный вход — это диагностическое сообщение и ненулевой код возврата.

1.2. СРОКИ

- Загрузка работы в репозиторий — до 23:59 25 октября.
- Защиты — 18 ноября.

1.3. ИНДИВИДУАЛЬНЫЙ ВАРИАНТ

Пусть s — номер студента в списке группы. Все параметры варианта вычисляются по s и приведены в описании соответствующей задачи. Номер варианта указывается в первой строке отчёта и выводится программой по ключу `--variant`.

Совпадение результатов у двух студентов с разными s означает, что вариант не учтён, и работа возвращается без защиты.

1.4. СОСТАВ СДАВАЕМОЙ РАБОТЫ

1. Репозиторий системы контроля версий с историей работы. Единственный коммит с полным решением — основание для дополнительных вопросов на защите, но не отказ в допуске.
2. Исходный код и скрипт запуска.
3. Файл `SOURCES.md` со списком внешних источников (см. п. 1.5.).
4. Отчёт. Требования к отчёту приведены отдельно для каждого задания. Отчёт — рабочий документ: в нем фиксируются принятые решения и числа, полученные на собственном варианте, а не пересказ лекций.
5. Собственный набор тестов: входные файлы для каждого режима каждой задачи, включая некорректные и вырожденные входы. Способ подготовки тестов не ограничен.
6. Файл `RESULTS.md` с выводом программы на собственных тестах. Числа в отчёте и в этом файле должны совпадать.

1.5. ЗАИМСТВОВАНИЯ И СИСТЕМЫ ГЕНЕРАЦИИ КОДА

Заимствование кода и текста не запрещено, но декларируется.

1. Каждый заимствованный или сгенерированный фрагмент отмечается в коде комментариями, обозначающими его начало и конец, и описывается в `SOURCES.md`: источник (ссылка, имя человека, название системы) и назначение фрагмента.
2. Само по себе декларирование баллов не снимает. Баллы снимает невозможность объяснить фрагмент на защите, независимо от того, откуда фрагмент взят и объявлен ли он.
3. Недекларированное заимствование — задание не засчитывается.
4. Если студент не может объяснить существенный фрагмент собственного кода, фрагмент считается недекларированным заимствованием со всеми последствиями п. 3. Существенность определяет преподаватель; ею обладают все фрагменты, реализующие содержание курса, и не обладают разбор аргументов командной строки, ввод-вывод и оформление.

1.6. ЗАЩИТА У ДОСКИ

Регламент защиты — 12 минут на всё задание. На защите разбирается одна из сданных задач, которую выбирает преподаватель.

1. **Закрытый тест** (3 минуты, 25% оценки). Преподаватель передаёт входные данные к выбранной задаче, которых студент не видел, и наблюдает запуск. Оценивается поведение программы, включая поведение на некорректном и вырожденном входе.
2. **Ручной счёт** (4 минуты, 25% оценки). Студент случайно вытягивает одну задачу из опубликованного списка, решает её на доске без компьютера и затем сверяет ответ с выводом собственной программы. Расхождение — предмет отдельного вопроса.
3. **Вопросы по собственному решению** (3 минуты, 30% оценки). Два вопроса из опубликованного банка и один вопрос вида «Покажи в своем коде, где...».

4. **Модификация на месте** (2 минуты, 20% оценки). Преподаватель называет изменение входных данных или одной строки кода. Студент сначала предсказывает, что изменится в выводе, затем вносит изменение и запускает программу. Оценивается предсказание, а не правка.

Пункты 2–4 проводятся и в том случае, когда программа не прошла закрытый тест.

Пересдача — один раз, с новым значением s, назначаемым преподавателем.

1.7. ОЦЕНИВАНИЕ

Критерий «задание». Для каждой задачи j по сданным файлам выставляется $q_j \in [0, 1]$ из двух частей:

- **программа** — все режимы задачи реализованы и работают на собственных тестах, выполнены требования п. 1.1.;
- **отчёт** — части отчёта, относящиеся к задаче, присутствуют полностью, числа совпадают с `RESULTS.md`.

Доли частей внутри задачи:

Задача	Программа	Отчёт
1.1	0,55	0,45
1.2	0,55	0,45
1.3	0,75	0,25

Оценка по критерию равна

$$G_{\text{зад}} = \sum_j w_j q_j,$$

где w_j — вес задачи внутри задания. Несданная задача даёт $q_j = 0$ и входит в сумму с нулём; перенормировки на число сданных задач нет.

Критерий «защита»

$$G_{\text{защ}} = 0{,}25\, g_1 + 0{,}25\, g_2 + 0{,}30\, g_3 + 0{,}20\, g_4,$$

где $g_i \in [0, 1]$ — результаты четырёх компонент регламента (п. 1.6.). Защита считается пройденной при $G_{\text{защ}} \geq 0{,}5$. Если защита не пройдена, задание не засчитывается: оба критерия равны нулю. Если не сдана ни одна задача, защита не проводится и оба критерия также равны нулю.

Перевод в итоговую оценку. Оба критерия выражаются числом от 0 до 1 и входят в итоговую оценку курса с весами, указанными в критериях оценивания курса.

2. Домашнее задание №1. Кодирование источников

Задание состоит из трёх связанных задач. Одна и та же модель источника проходит через все три: сначала она моделируется, затем кодируется равномерным кодом, затем сравнивается с реальным сжатием. Веса задач: задача 1.1 — 30%, задача 1.2 — 35%, задача 1.3 — 35%.

ПАРАМЕТРЫ ВАРИАНТА

По номеру s вычисляются:

1. $k = 5 - (s \bmod 3)$ — мощность алфавита источника;
2. $M = 2 + (s \bmod 3)$ — число моделей в описании источника;
3. $L = 3 + (s \bmod 5)$ — длина списка `source`;
4. $d = 1 + (s \bmod 3)$ — глубина памяти в расширенном формате;
5. q — элемент с номером $s \bmod 4$ набора $(2, 3, 5, 7)$ — мощность алфавита кодера;
6. $\varepsilon = 10^{-(2+(s \bmod 3))}$ — допустимая вероятность ошибки декодирования.

Все примеры источников, приводимые в отчёте, строятся на алфавите мощности k с M моделями и списком `source` длины L.

ЗАДАЧА 1.1. МОДЕЛИРОВАНИЕ ДИСКРЕТНЫХ ИСТОЧНИКОВ

Требуется реализовать программу, моделирующую работу дискретного источника.

Формат описания источника

ПРИМЕР 1

Формат файла с описанием дискретного источника (формат называется json):

```
{
    "models": {
        "имя модели": {
            "символ": "вероятность появления",
        },
    },
    "switches": {
        "имя переключателя": {
                "имя модели": "вероятность",
        },
    },
    "source": [
        "имя переключателя", "имя переключателя",
    ]
}
```

ПРИМЕР 2

Приведём пример файла:

```
{
    "models": {
        "монета_1": {
            "0": "1/3",
            "1":  "2/3"
        },
        "монета_2": {
            "0": "2/3",
            "1": "1/3"
        }
    },
    "switches":{
        "switch_1": {
            "монета_1": "1/2",
            "монета_2": "1/2"
        },
        "switch_2": {
            "монета_1": "0.3",
            "монета_2": "0.7"
        },
        "switch_3": {
            "монета_1": "0.4",
            "монета_2": "0.6"
        }
    },
    "source": ["switch_1", "switch_2", "switch_3", "switch_2"]
}
```

Список `source` задаёт циклическую последовательность переключателей. На такте i выбирается переключатель с номером $i \bmod L$ в этом списке, по его распределению выбирается модель, по модели — символ. Выборы на разных тактах независимы.

Вероятности записаны строками: допускаются обыкновенные дроби и десятичная запись. Программа обязана работать с точной рациональной арифметикой и отвергать описание, в котором распределение не нормировано, упомянута несуществующая модель или переключатель, встречается отрицательная вероятность либо пустой список `source`.

Режим 1. Реализация источника

На вход подаются файл с описанием источника и число N либо значение `None`.

При $N > 0$ программа выдаёт реализацию источника длины N символов. В остальных случаях выдача продолжается до прерывания пользователем. Способ прерывания описывается в отчёте; кроссплатформенный перехват отдельной клавиши не требуется.

Режим 2. Точная вероятность и частотная оценка

На вход подаются файл с описанием источника, число $N > 0$ и последовательность символов $a_1, a_2, \ldots, a_r$.

Программа выводит три величины.

1. Точное значение $\Pr\{X_1 = a_1, \dots, X_r = a_r\}$, вычисленное по модели в рациональной арифметике.
2. Точное значение $\frac{1}{L}\sum_{j=0}^{L-1} \Pr\{X_{j+1} = a_1, \dots, X_{j+r} = a_r\}$ — среднее по позициям внутри периода.
3. Частотную оценку по одной реализации длины N: долю позиций $i = 1, \dots, N - r + 1$, для которых $X_i \dots X_{i+r-1} = a_1 \dots a_r$; знаменатель: $N - r + 1$.

Дополнительно программа выводит таблицу частотной оценки для $N = 10^2, 10^3, \dots, 10^6$ и модуль её отклонения от каждой из двух точных величин.

В отчёте требуется указать, к какой из двух точных величин сходится частотная оценка и почему. Это — основной содержательный пункт задачи.

Расширение формата

Требуется предложить расширение формата, единое для источников всех типов, в котором выразимы:

- марковский источник глубины d над алфавитом мощности k;
- стационарный неэргодический источник;
- исходный формат как частный случай без изменения уже написанных файлов.

Отдельно отметим: в исходном формате символы на разных тактах независимы, поэтому стационарный неэргодический источник в нём невыразим. Обоснование этого утверждения и конструкция, снимающая ограничение, — предмет вопросов на защите.

Программа обязана поддерживать расширенный формат во всех режимах и принимать файлы исходного формата без конвертации. Спецификация расширения (одна–две страницы: грамматика, семантика, примеры) прикладывается к работе отдельным файлом.

ЗАДАЧА 1.2. РАВНОМЕРНОЕ КОДИРОВАНИЕ ИСТОЧНИКА БЕЗ ПАМЯТИ

Постановка

Источник без памяти задан распределением P на алфавите A, $|A| = k$. Блок длины n кодируется словом длины ℓ в алфавите мощности q. Скорость кода $R = \dfrac{\ell \log_2 q}{n}$ бит на символ источника. Ошибка декодирования наступает, когда блок не попал в кодируемое множество; её вероятность обозначим $P_e(n)$.

Режим 1

На вход подаётся описание источника. Программа проверяет, что источник стационарен и не имеет памяти, и выводит его собственную энтропию. Если источник не удовлетворяет условию, программа указывает, какое именно условие нарушено.

Режим 2

На вход подаются описание источника, скорость R, граница ε и мощность q.

Программа выводит:

- таблицу $P_e(n)$ для $n = 1, \dots, n_{\max}$ при $\ell(n) = \lfloor nR / \log_2 q \rfloor$, где множество кодируемых блоков строится оптимально — из q^ℓ наиболее вероятных блоков;
- минимальное n, при котором $P_e(n) \le \varepsilon$, либо сообщение об отсутствии такого n на рассмотренном отрезке;
- ответ на вопрос, существует ли n_0 такое, что $P_e(n) \le \varepsilon$ при всех $n \ge n_0$, и обоснование ответа через сравнение R и $H(X)$;
- описание построенного кода в файле: формат определяет разработчик.

Заметим, что при $R < H(X)$ условие $P_e(n) \le \varepsilon$ может выполняться при малых n и обязательно нарушается при больших. Ответ «кода не существует» при $R < H(X)$ без оговорки о малых n считается ошибкой.

Требование к сложности

Программа обязана отрабатывать при $k \le 5$ и $n_{\max} = 50$ не более чем за 10 секунд. Перебор всех k^n блоков при таких параметрах невозможен, поэтому требуется метод, опирающийся на структуру задачи. Выбранный метод и его сложность обосновываются в отчёте и защищаются у доски.

Переполнение и потеря точности при вычислении вероятностей блоков — отдельное проектное решение, которое также обосновывается.

ЗАДАЧА 1.3. АРХИВАТОР ZIPMEHUFFMAN

Режимы

Сжатие. На вход подаётся бинарный файл. Программа выполняет двухпроходное сжатие методом Хаффмана и сохраняет результат в файл с тем же именем и расширением `zmh`. На первом проходе строится модель, на втором выполняется кодирование.

Восстановление. На вход подаётся файл с расширением `zmh`. Программа восстанавливает исходный файл и проверяет целостность результата.

Формат `zmh`

Формат контейнера определяет разработчик. Обязательные свойства:

- сигнатура, позволяющая отличить `zmh` от файла чужого формата;
- контрольная сумма исходных данных, позволяющая обнаружить повреждение;
- корректная обработка неполного последнего байта потока кодов;
- корректная работа на пустом файле и на файле, состоящем из повторений одного байта;

- описание модели, хранящееся компактно; оценка накладных расходов в байтах приводится в отчёте.

Повреждённый файл, файл чужого формата и обрезанный файл различаются в диагностике.

Измерения

Программа по ключу `--stats` выводит для сжимаемого файла: эмпирическую энтропию H на байт, среднюю длину кодового слова $\bar{\ell}$, размер заголовка в байтах, размер данных в байтах и фактический коэффициент сжатия.

В отчёте приводится таблица этих величин не менее чем для пяти файлов: текст на русском языке, текст на английском языке, изображение в сжатом формате, файл из повторяющегося байта, пустой файл. Расхождение между $\bar{\ell}$ и H и случаи роста размера файла объясняются.

2.1. ТОЧКИ ПРОЕКТНЫХ РЕШЕНИЙ

Ниже перечислены решения, которые студент принимает сам и защищает у доски. Каждое описывается в отчёте абзацем: что выбрано, какие еще варианты рассматривались, по какому критерию сделан выбор.

1. Расширение формата описания источников на память и на неэргодичность.
2. Представление вероятностей и арифметика вычислений.
3. Метод построения множества наиболее вероятных блоков в задаче 1.2 и борьба с переполнением.
4. Структура заголовка `zmh` и способ хранения модели.
5. Способ обнаружения повреждений и разграничение диагностик.

2.2. ОТЧЁТ

Отчёт содержит:

- номер варианта и вычисленные по нему параметры;
- пять абзацев по п. 2.1.;
- спецификацию расширенного формата и в ней — по одному примеру стационарного, нестационарного, эргодического и неэргодического источника на алфавите мощности k с указанием, какое свойство обеспечивает каждая конструкция;
- таблицы измерений из задач 1.1 и 1.3 с числами собственного варианта и фактическое время работы задачи 1.2;
- журнал неудач: что было реализовано и отброшено, по какой причине.

Объём отчёта не оценивается. Отчёт служит источником вопросов на защите: утверждение, не подтверждённое на доске, снимает баллы по п. 3 регламента.

2.3. ЗАДАЧИ РУЧНОГО СЧЁТА

На защите вытягивается одна задача.

1. Вычислить энтропию источника с тремя символами и вероятностями, заданными обыкновенными дробями, с точностью до $0{,}01$ бита. Указать, при каком распределении на этом алфавите энтропия максимальна.
2. Построить код Хаффмана для шести заданных весов, вычислить среднюю длину и сравнить её с энтропией. Предъявить два различных дерева и объяснить, почему средняя длина у них совпадает.
3. Для двоичного источника и $n = 3$ выписать все типы блоков, их вероятности и мощности. Отобрать 4 наиболее вероятных блока и вычислить P_e.
4. По источнику из примера 2 выписать распределение символа на такте $L + 2$ и вычислить точную вероятность заданного блока длины два, начинающегося на первом такте.
5. Для заданного источника указать границу на R, отделяющую случай $P_e(n) \to 0$ от случая $P_e(n) \to 1$, и объяснить переход.
6. Для заданного набора частот байтов оценить размер заголовка `zmh` своего формата и указать, при каком размере входного файла сжатие уходит в минус.

2.4. БАНК ВОПРОСОВ

1. К какой величине сходится частотная оценка из режима 2 задачи 1.1 и почему не к вероятности блока на первом такте?
2. Покажи в своём расширенном формате запись стационарного неэргодического источника и объясни, какая именно конструкция лишает его эргодичности.
3. Почему в исходном формате примера 1 неэргодический источник невыразим?
4. Почему вероятности заданы строками, а не числами? Что сломается при использовании чисел с плавающей точкой?
5. Какую структуру задачи ты используешь, чтобы не перебирать k^n блоков? Оцени сложность своего метода.
6. Как ты вычисляешь вероятность блока, не теряя точности? Покажи это место в коде.
7. При $R < H(X)$ твоя программа нашла подходящее n. Противоречит ли это обратной теореме кодирования?
8. Сколько байт занимает заголовок `zmh` на твоём тестовом файле и из чего он складывается?
9. Как выглядит заголовок для файла из одного повторяющегося байта? Какова при этом длина кодового слова?
10. Как программа отличает повреждённый `zmh` от файла чужого формата? Что произойдёт при инверсии одного бита в середине сжатых данных?
11. Назови $\bar{\ell}$ и H для своего текстового теста и объясни разрыв между ними.